

# 同济大学计算机系

## 计算机组成原理实验报告



学 号 2454307

姓 名 汤皓宇

专 业 信息安全

授课老师 郭玉臣

# 一、实验内容

32 位有符号除法器、32 位无符号除法器

# 二、硬件逻辑图

未要求

# 三、模块建模

## 1. 32 位无符号除法器

该模块实现了一个 32 位无符号整数硬件除法器，采用移位相减（非恢复余数法，Non-Restoring Division）迭代算法完成运算。模块接收 32 位被除数（dividend）和除数（divisor），在 start 信号触发后进入忙碌状态（busy = 1），每个时钟周期执行一次"移位 + 试减/试加"操作：将余数寄存器 reg\_r 与商寄存器 reg\_q 的最高位拼接后，根据上一轮结果的符号位（reg\_sign）决定是做减法还是加法（非恢复法的核心），并将结果的符号位取反后移入商寄存器的最低位；经过 32 个时钟周期后（count 计数到 31），运算完成，busy 清零，q 输出商，r 输出余数（若最终余数为负则自动加回除数以恢复正确余数）。

```
module DIVU(  
    input [31:0]dividend,  
    input [31:0]divisor,  
    input start,  
    input clock,  
    input reset,  
    output [31:0]q,  
    output [31:0]r,  
    output busy  
);  
    reg[4:0] count;  
    reg[31:0] reg_q;  
    reg[31:0] reg_r;  
    reg reg_busy;  
    reg reg_sign;  
  
    wire [32:0] sub_add = reg_sign ?  
        ({reg_r, reg_q[31]} + {1'b0, divisor}):  
        ({reg_r, reg_q[31]} - {1'b0, divisor});  
  
    assign q = reg_q;  
    assign r = reg_sign ? reg_r + divisor : reg_r;  
    assign busy = reg_busy;
```

```

always @(posedge clock) begin
    if (reset) begin
        count <= 5'b0;
        reg_busy <= 0;
        reg_sign <= 0;
    end else begin
        if (start) begin
            count <= 5'b0;
            reg_q <= dividend;
            reg_r <= 32'b0;
            reg_busy <= 1;
            reg_sign <= 0;
        end else if (busy) begin
            reg_r <= sub_add[31:0];
            reg_sign <= sub_add[32];
            reg_q <= {reg_q[30:0], ~sub_add[32]};
            count <= count + 1;
            if (count == 5'd31) reg_busy <= 0;
        end
    end
end
end
endmodule

```

## 2. 32 位有符号除法器

该模块实现了一个 32 位有符号整数硬件除法器, 在 DIVU 无符号除法器的基础上增加了符号处理逻辑。模块在运算开始时 (**start** 触发) 首先对被除数和除数取绝对值 (通过按位取反加一的补码操作), 并预先计算商的符号 (**reg\_q\_sign**, 由两操作数符号异或得出, 即同号为正、异号为负) 和余数的符号 (**reg\_r\_sign**, 与被除数同号, 符合有符号除法的余数规则); 随后与 DIVU 相同, 用 32 个时钟周期的非恢复余数法对两个绝对值进行无符号迭代除法; 运算完成后, 输出端根据预存的符号位对商和余数分别进行条件取反 (补码还原), 从而得到符合 截断除零 (**truncated division**) 语义的有符号商 **q** 和余数 **r**。

```

module DIV(
    input [31:0]dividend,
    input [31:0]divisor,
    input start,
    input clock,
    input reset,
    output [31:0]q,
    output [31:0]r,
    output busy
);
    reg[4:0] count;

```

```

    reg[31:0] reg_q;
    reg reg_q_sign;
    reg[31:0] reg_r;
    reg reg_r_sign;
    reg reg_busy;
    reg reg_sign;
    wire [31:0] abs_dividend = dividend[31] ? ~dividend + 1 :
dividend;
    wire [31:0] abs_divisor = divisor[31] ? ~divisor + 1 : divisor;

    wire [32:0] sub_add = reg_sign ?
        ({reg_r, reg_q[31]} + {1'b0, abs_divisor}):
        ({reg_r, reg_q[31]} - {1'b0, abs_divisor});

    wire [31:0] true_r_mag = reg_sign ? (reg_r + abs_divisor) :
reg_r;
    assign q = reg_q_sign ? (~reg_q + 1'b1) : reg_q;
    assign r = reg_r_sign ? (~true_r_mag + 1'b1) : true_r_mag;
    assign busy = reg_busy;

    always @(posedge clock) begin
        if (reset) begin
            count <= 6'b0;
            reg_busy <= 0;
        end else begin
            if (start) begin
                count <= 0;
                reg_q <= abs_dividend;
                reg_r <= 0;
                reg_q_sign <= dividend[31] ^ divisor[31];
                reg_r_sign <= dividend[31];
                reg_sign <= 0;
                reg_busy <= 1;
            end else if (busy) begin
                reg_r <= sub_add[31:0];
                reg_sign <= sub_add[32];
                reg_q <= {reg_q[30:0], ~sub_add[32]};
                count <= count + 1;
                if (count == 5'd31) reg_busy <= 0;
            end
        end
    end
end
endmodule

```

## 四、测试模块建模

### 1. 32 位无符号除法器

该模块是针对 DIVU 无符号除法器的 Verilog 仿真测试平台 (Testbench)，用于验证 DIVU 模块的运算正确性。实现上，模块例化 DIVU 作为被测单元(UUT)，生成周期为 10ns 的时钟信号，并封装了一个 do\_div 任务来规范化每次除法测试的流程：先设置操作数，再在时钟边沿拉高 start 脉冲一个周期以触发运算，然后通过 wait(!busy) 阻塞等待 32 个周期的运算完成，最后用 \$display 同时打印实际输出的商和余数与 Verilog 内置运算符 / 和 % 计算的期望值进行比对。测试用例覆盖了多种边界场景：常规除法 ( $100 \div 7$ )、零被除数 ( $0 \div 1$ )、被除数等于除数 ( $1 \div 1$ )、32 位最大值自除 ( $0xFFFFFFFF \div 0xFFFFFFFF$ )、大数除以小数 ( $0xFFFFFFFF \div 3$ ) 以及较大整数除法 ( $1000000 \div 999$ )，从而全面检验 DIVU 模块在典型及极端输入下的功能正确性。

```
module DIVU_tb;
    reg [31:0] dividend, divisor;
    reg start, clock, reset;
    wire [31:0] q, r;
    wire busy;

    DIVU uut (
        .dividend(dividend), .divisor(divisor),
        .start(start), .clock(clock),
        .reset(reset), .q(q), .r(r), .busy(busy)
    );

    // 时钟: 10ns 周期
    initial clock = 0;
    always #5 clock = ~clock;

    // 等待运算完成的任务
    task do_div;
        input [31:0] a, b;
        begin
            dividend = a; divisor = b;
            @(posedge clock); #1;
            start = 1;
            @(posedge clock); #1;
            start = 0;
            wait(!busy);
            @(posedge clock); #1;
            $display("DIVU: %0d / %0d = q=%0d, r=%0d (expect q=%0d, r=%0d)",
                a, b, q, r, a/b, a%b);
        end
    endtask
endmodule
```

```

        end
    endtask

    initial begin
        reset = 1; start = 0; dividend = 0; divisor = 1;
        @(posedge clock); #1;
        reset = 0;

        do_div(100,      7);
        do_div(0,        1);
        do_div(1,        1);
        do_div(32'hFFFF_FFFF, 32'hFFFF_FFFF); // 最大值 / 最大值
        do_div(32'hFFFF_FFFF, 3);
        do_div(1000000,  999);

        $finish;
    end
endmodule

```

## 2. 32 位有符号除法器

该模块是针对 DIV 有符号除法器的 Verilog 仿真测试平台(Testbench), 与 DIVU\_tb 结构基本一致, 但重点在于验证有符号运算的正确性。do\_div 任务的输入参数声明为 signed [31:0], 输出显示时统一使用 \$signed() 进行有符号解释, 并以 Verilog 内置的有符号运算符 / 和 % 作为参考期望值。测试用例系统性地覆盖了有符号除法的四种符号组合 (+/+、-/+、+/-、-/-), 以及边界情况:  $1 \div 1$ 、 $-1 \div 1$ , 以及 32 位最小负数 0x80000000 (即 -2147483648) 除以 1——后者是有符号整数运算中的经典溢出边界场景, 用于检验 DIV 模块在补码极端值下的符号处理和绝对值转换逻辑是否健壮。

```

module DIV_tb;
    reg [31:0] dividend, divisor;
    reg start, clock, reset;
    wire [31:0] q, r;
    wire busy;

    DIV uut (
        .dividend(dividend), .divisor(divisor),
        .start(start), .clock(clock),
        .reset(reset), .q(q), .r(r), .busy(busy)
    );
    initial clock = 0;
    always #5 clock = ~clock;
    task do_div;
        input signed [31:0] a, b;
        begin
            dividend = a; divisor = b;

```

```

        @(posedge clock); #1;
        start = 1;
        @(posedge clock); #1;
        start = 0;
        wait(!busy);
        @(posedge clock); #1;
        $display("DIV: (%0d) / (%0d) = q=%0d, r=%0d    (expect
q=%0d, r=%0d)",
                $signed(a), $signed(b),
                $signed(q), $signed(r),
                $signed(a) / $signed(b),
                $signed(a) % $signed(b));

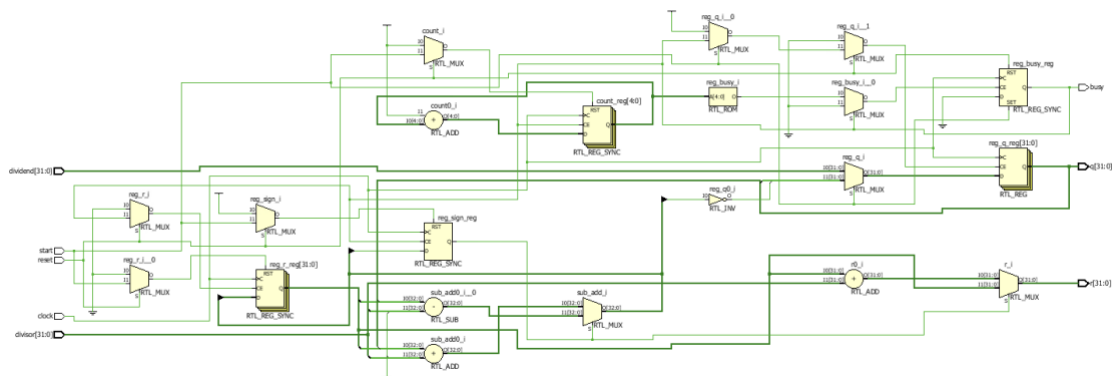
    end
endtask
initial begin
    reset = 1; start = 0; dividend = 0; divisor = 1;
    @(posedge clock); #1;
    reset = 0;
    do_div( 100,  7);    // + / +
    do_div(-100,  7);    // - / +
    do_div( 100, -7);    // + / -
    do_div(-100, -7);    // - / -
    do_div(  1,  1);
    do_div(-1,   1);
    do_div(32'h8000_0000, 1); // 最小负数 / 1

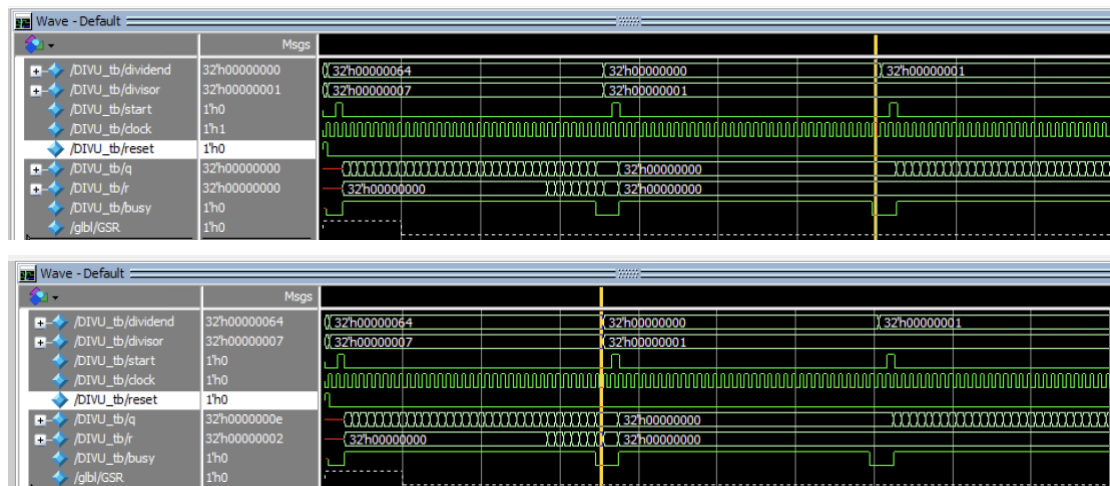
    $finish;
end
endmodule

```

## 五、实验结果

### 1. 32 位无符号除法器





## 2. 32 位有符号除法器

